With our first Milestone coming to a close, I wanted to document my findings on what I enjoyed, what I struggled with, and how I can improve in the future when continuing this assignment and other Unity projects I choose to create.

By analysing my own prototype as a piece of interactive media, I can observe my player being able to run, walk, jump, and crouch in a digital play space. This alone shows how the body that inhabits the play space is now turned into a “surrogate body” as players master the controls of their character. The identity shifts from observing an object and switches to taking ownership of a digital vessel. How this relates to the play of the character is that in the game I’m creating, movement is the core gameplay; with that being said, players, through the controls they are given, should feel immersed as every button press has an instant reaction to the character on screen.

The core engine systems I used were CineMachine, New Input System, and ProBuilder.

To start, ProBuilder made building a level easy, at first, it felt intimidating with the different menus and buttons. Understanding the pattern of length, width, and height to create an object felt strange at first; however, after a few times of doing the same pattern, it became fluent in building blocks and other shapes to create a test level.

After I learned the New Input System. This system made the coding of controls really convenient, as it would automatically link the creation of new controls to a key-bind the player set in an easy-to-read submenu. Instead of taking painstaking hours of code to program each button to a certain control, a few minutes of clicking into a menu really helped with time and effort to create responsive controls.

Finally, Cinemachine. The default cameras are very difficult to understand in Unity, especially if you want a certain point of view and, just like the New Input System, being able to create that point of view with just a few clicks instead of coding really saved me some time and struggle. With that, I ended up with a third-person camera that is not only dynamic but unique too. There are issues I do want to fix about how the camera functions; however, it'll be in the next milestone.

The most difficult technical hurdle I faced was coding the jump function into my game. As movement is the core function of my game, I wanted all the animations to be smooth and responsive; being able to jump is essential to that factor of play. When adding the jump, I followed my code for sprinting, adding the new input and code for said input. However, when it came time to launch the player up, I had to think about the Y axis of my game and how high the player jumped. After watching a few tutorials and forum posts, I got the player to jump off the ground. However, that was the start of the most difficult problem I faced.

When the player jumped, it was still playing the idle or run animations. My game is all about fluid movement, so I needed to add jump animations to my character. I devised the animation into 3 parts: beginning jump, fall, and land, so I can also optimize if the player decides to run off a platform; it'll skip the beginning jump animation and go straight to the falling one. This is when I ran into my first issue: when the player would hit the floor after jumping, the player would be stuck in the falling animation. After some research, I found the best solution was to make a boolean that detects if the player is on a

layer that I made called ground (I have to manually label platforms as ground for the model to detect the floor), and therefore the landing animation would play and switch back to the default animations.

My second problem was when I noticed that when I jumped, the player would start floating upwards and never come down. This was an animation error, as the animations I used had the animation moving the entire model upwards. After learning the Inspector for animation properties, I was able to fix this issue.

Finally, my final issue was the beginning jump animation. I noticed that when I jumped, the animation would play when the player was already halfway in the air. I started fixing this by adding a new trigger to detect the button press and then delayed the jump so it would sync the animation right when the model would push off the floor. It didn't work, as now the model would repeat the same few frames of the jump animation while being halfway through the air. After a few hours of troubleshooting, I analysed the frames it was playing and noticed they weren't the wind-up of the jump, but the frames where the player should go airborne. This made me realise that the animation was playing correctly; something was keeping it on loop. That made me notice the other conditions I had were interfering with the trigger condition. After I deleted the other conditions, the jump finally worked smoothly.

Real-time response is very important to my game, and in the case of Steve Swink, his reading establishes the baseline definition of game feel being ".Real-time control of virtual objects in a simulated space, with interactions emphasized by polish". Swink's point of interest was that controlling an object in a blank void lacks spatial context and physical meaning. The aspects of Real-time controls, a simulated space, and polish help to design a fully responsive and immersive game.

A gray box design gives the context of all these factors by testing the "Raw Game Feel", as the real-time controls and simulated space are the main focus. This focus allows us to isolate if all the player inputs feel good and collision geometry functions correctly. When these factors are perfect we can focus on the polished game (adding texture, particles, decorations, and story).

The Unity can be strange at times, where I found the most struggle is with the physics, the gravity either felt too soft or too floaty at time; however, instead of changing ideas, I adapted to this underlying architecture by leaning into the physics and added mid air steering so players can use the odd physics to their advantage.